Import Libraries

```
import matplotlib.pyplot as plt
import seaborn as sns

import numpy as np
import tensorflow as tf
from tensorflow.keras import Sequential,layers,Model
from tensorflow.keras.layers import Dense
import os

import keras
from tensorflow.keras.regularizers import l1,l2
```

Load Dataset

```
data = '/content/drive/MyDrive/Colab Notebooks/week 6/cifar-10-python.tar.gz'
```

Extracting Tar File

```
import tarfile
if data.endswith("tar.gz"):
    tar = tarfile.open(data, "r:gz")
    tar.extractall()
    tar.close()

path = '/content/cifar-10-batches-py'
os.listdir(path)
train_batch = []
test_batch = os.path.join(path,'test_batch')
for i in os.listdir(path):
  if 'data_batch' in i:
    train_batch.append(os.path.join(path,i))
train_batch ,test_batch
```

```
(['/content/cifar-10-batches-py/data_batch_5',
  '/content/cifar-10-batches-py/data_batch_4',
  '/content/cifar-10-batches-py/data_batch_1',
  '/content/cifar-10-batches-py/data_batch_2',
  '/content/cifar-10-batches-py/data_batch_3'],
 '/content/cifar-10-batches-py/test_batch')
```

```
import pickle
import sys
from keras import backend as K
from tensorflow.keras.utils import to_categorical

def load_batch(fpath, label_key='labels'):

  with open(fpath, 'rb') as f:
      if sys.version_info < (3,):
```

```
            d = pickle.load(f)
        else:
            d = pickle.load(f, encoding='bytes')

            d_decoded = {}
            for k, v in d.items():
                d_decoded[k.decode('utf8')] = v
            d = d_decoded
    data = d['data']
    labels = d[label_key]

    data = data.reshape(data.shape[0], 3, 32, 32)
    return data, labels


def load_data():
    num_train_samples = 50000
    x_train = np.empty((num_train_samples, 3, 32, 32), dtype='uint8')
    y_train = np.empty((num_train_samples,), dtype='uint8')

    for i in range(1, 6):
        fpath = os.path.join(path, 'data_batch_' + str(i))
        (x_train[(i - 1) * 10000: i * 10000, :, :, :],
        y_train[(i - 1) * 10000: i * 10000]) = load_batch(fpath)
    fpath = os.path.join(path, 'test_batch')
    x_test, y_test = load_batch(fpath)

    y_train= np.reshape(y_train, (len(y_train), 1))
    y_test = np.reshape(y_test, (len(y_test), 1))

    if K.image_data_format() == 'channels_last':
        x_train = x_train.transpose(0, 2, 3, 1)
        x_test = x_test.transpose(0, 2, 3, 1)

    return (x_train, y_train), (x_test, y_test)

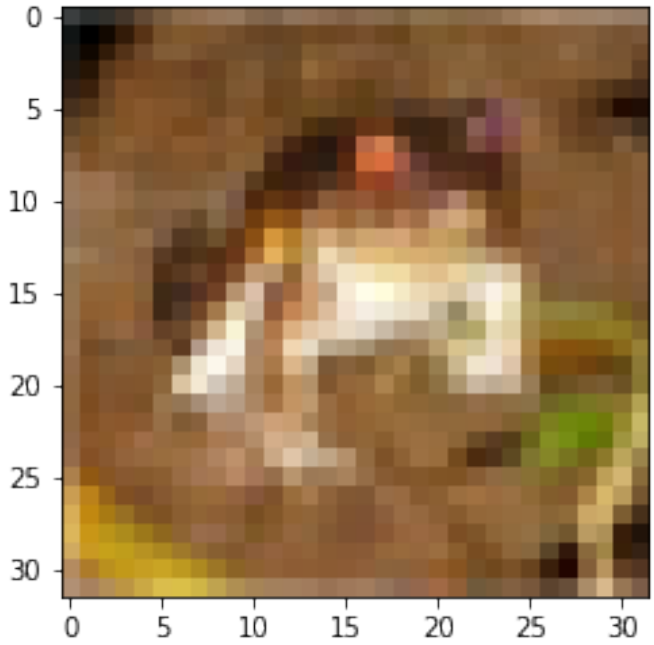
(train_img,train_label),(test_img,test_label) = load_data()
train_img.shape,train_label.shape,test_img.shape,test_label.shape
```

```
((50000, 32, 32, 3), (50000, 1), (10000, 32, 32, 3), (10000, 1))
```

Testing Images

```
def img_show(img):
    plt.imshow(img)
    plt.show();

img_show(train_img[0])
```

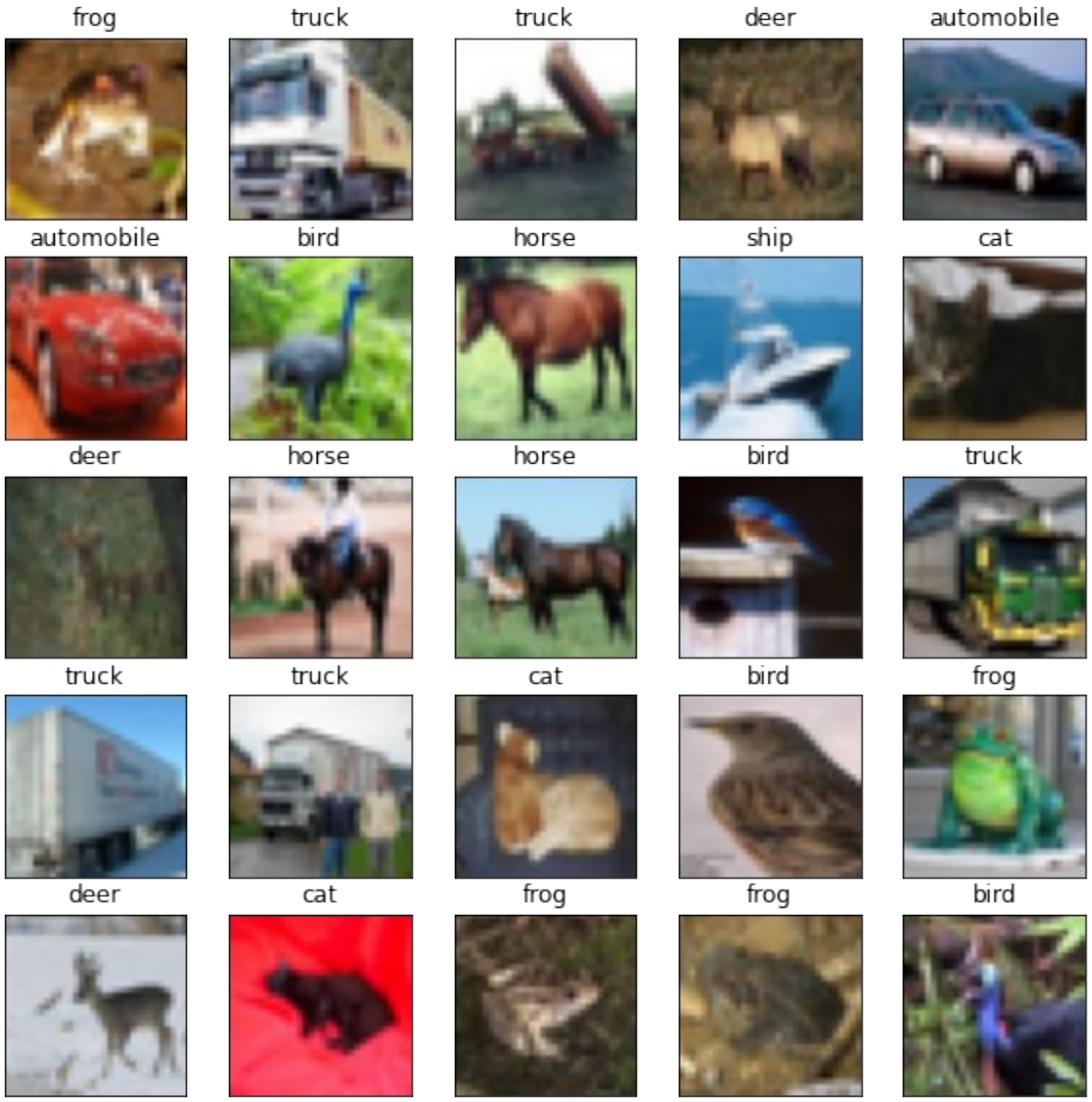



```
label_names = {
    0:'airplane',
    1:'automobile',
    2:'bird',
    3:'cat',
    4:'deer',
    5:'dog',
    6:'frog',
    7:'horse',
    8:'ship',
    9:'truck'
}
train_label[0][0]
```

```
6
```

```
rows, columns = 5, 5

fig = plt.figure(figsize=(10,10))
for i in range(1,columns*rows+1):
  fig.add_subplot(rows, columns, i)
  plt.imshow(train_img[i-1])
  plt.xticks([])
  plt.yticks([])
  plt.title("{}".format(label_names[train_label[i-1][0]]))
plt.show();
```

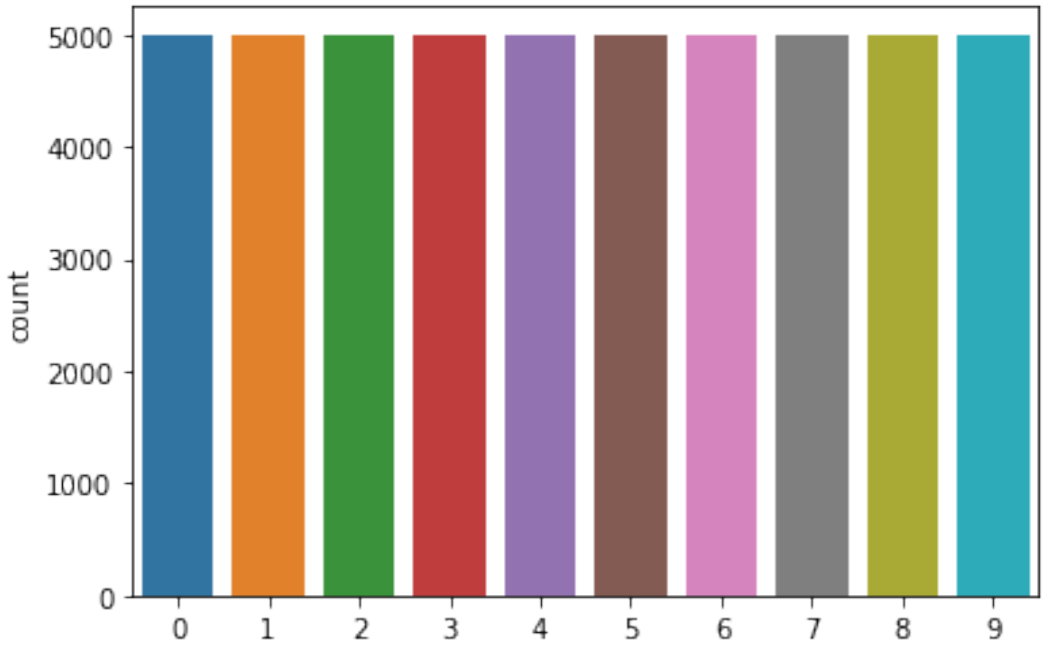


Checking if the data is balanced or not

```
train =train_label.reshape(-1)
sns.countplot(train)
```

```
/usr/local/lib/python3.7/dist-packages/seaborn/_decorators.py:43:
FutureWarning: Pass the following variable as a keyword arg: x. From
version 0.12, the only valid positional argument will be `data`, and
passing other arguments without an explicit keyword will result in an
error or misinterpretation.
  FutureWarning

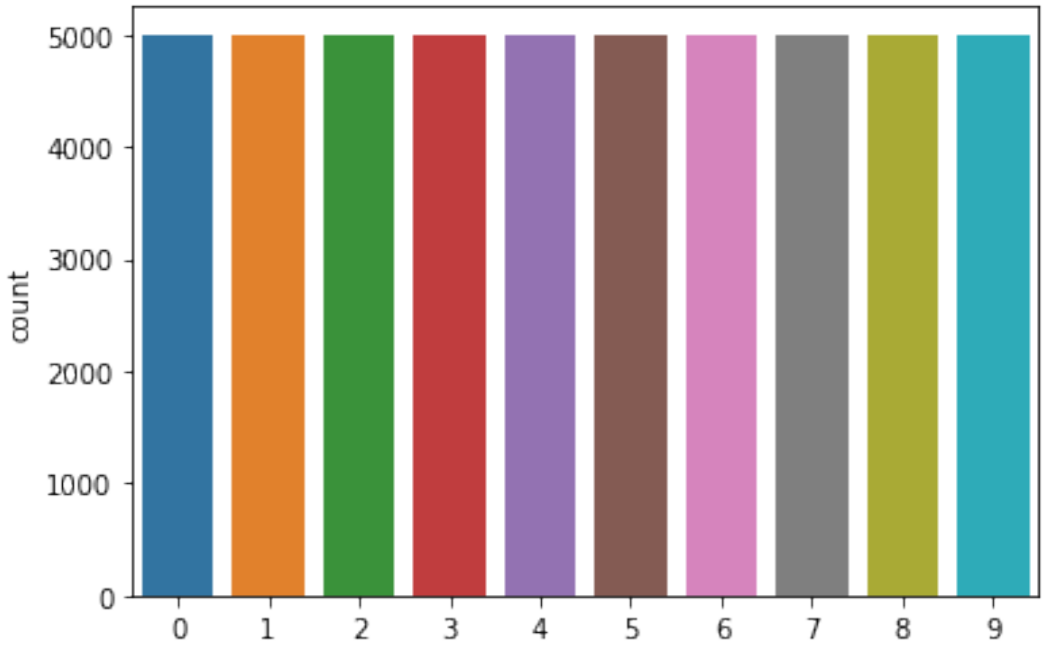
<matplotlib.axes._subplots.AxesSubplot at 0x7fec444f10d0>
```




```
test =train_label.reshape(-1)
sns.countplot(test)
```

```
/usr/local/lib/python3.7/dist-packages/seaborn/_decorators.py:43:
FutureWarning: Pass the following variable as a keyword arg: x. From
version 0.12, the only valid positional argument will be `data`, and
passing other arguments without an explicit keyword will result in an
error or misinterpretation.
  FutureWarning
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fec435cbfd0>
```




```
unique, counts = np.unique(train, return_counts=True)

print (np.asarray((unique, counts)).T)
```

```
[[   0 5000]
 [   1 5000]
 [   2 5000]
 [   3 5000]
 [   4 5000]
 [   5 5000]
 [   6 5000]
 [   7 5000]
 [   8 5000]
 [   9 5000]]
```

```
unique, counts = np.unique(test, return_counts=True)

print (np.asarray((unique, counts)).T)
```

```
[[   0 5000]
 [   1 5000]
 [   2 5000]
 [   3 5000]
 [   4 5000]
 [   5 5000]
 [   6 5000]
 [   7 5000]
 [   8 5000]
 [   9 5000]]
```

The Dataset is balanced

Data Normalization

```
train_img = train_img/255.0
test_img = test_img/255.0
train_img.shape,test_img.shape
```

```
((50000, 32, 32, 3), (10000, 32, 32, 3))
```

```
train_label.shape,test_label.shape
```

```
((50000, 1), (10000, 1))
```

```
train_label = tf.keras.utils.to_categorical(train_label)
test_label = tf.keras.utils.to_categorical(test_label)

train_label.shape,test_label.shape
```

```
((50000, 10), (10000, 10))
```

Data Augmentation

```
# from tensorflow.keras.preprocessing.image import ImageDataGenerator

# datagen = ImageDataGenerator(

# rotation_range = 40,
# width_shift_range = 0.2,
# height_shift_range = 0.2,
# shear_range = 0.2,
# zoom_range = 0.2,
# horizontal_flip = True,
# validation_split=0.2

# )

# train_generator = datagen.flow(train_img,train_label,batch_size =
32,subset='training')

# validation_generator = datagen.flow(train_img,train_label,batch_size
= 32,subset='validation')

stop_early = tf.keras.callbacks.EarlyStopping(monitor =
'val_accuracy',mode = 'auto', patience = 5)
tb_callback =
tf.keras.callbacks.TensorBoard(log_dir="/content/drive/MyDrive/Colab
Notebooks/week 6/logs2", histogram_freq=1)
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss',
factor = 0.2, patience = 5,mode = 'auto', min_lr = 0.0001)
callbacklist = [stop_early,tb_callback,reduce_lr]
```

Model

```
weight_decay = 1e-4
batch_size = 64
initial_epochs = 10
lr = 0.001
lr_decay = 1e-6

def model_build():
  inputs = keras.Input(shape=(32, 32, 3))
  x = tf.keras.layers.Conv2D(filters = 32,
                                        kernel_size = (3,3),
                                        padding = 'same',
                                        activation = 'relu',
                                        kernel_regularizer =
l2(weight_decay)
                                        )(inputs)
  x = tf.keras.layers.BatchNormalization()(x)
  x = tf.keras.layers.Conv2D(filters = 32,
                                        kernel_size = (3,3),
                                        padding = 'same',
                                        activation = 'relu',

                                        kernel_regularizer =
l2(weight_decay)
                                        )(x)
  x = tf.keras.layers.BatchNormalization()(x)
  x = tf.keras.layers.MaxPool2D(pool_size=(2,2))(x)
  x = tf.keras.layers.Dropout(rate = 0.2)(x)

  x = tf.keras.layers.Conv2D(filters = 64,
                                        kernel_size = (3,3),
                                        padding = 'same',
                                        activation = 'relu',

                                        kernel_regularizer =
l2(weight_decay)
                                        )(x)
  x = tf.keras.layers.BatchNormalization()(x)
  x = tf.keras.layers.Conv2D(filters = 64,
                                        kernel_size = (3,3),
                                        padding = 'same',
                                        activation = 'relu',

                                        kernel_regularizer =
l2(weight_decay)
                                        )(x)

  x = tf.keras.layers.BatchNormalization()(x)

  x = tf.keras.layers.MaxPool2D(pool_size=(2,2))(x)
```

```
  x = tf.keras.layers.Dropout(rate = 0.3)(x)

  x = tf.keras.layers.Conv2D(filters = 128,
                                       kernel_size = (3,3),
                                       padding = 'same',
                                       activation = 'relu',

                                       kernel_regularizer =
l2(weight_decay)
                                       )(x)

  x = tf.keras.layers.BatchNormalization()(x)
  x = tf.keras.layers.Conv2D(filters = 128,
                                       kernel_size = (3,3),
                                       padding = 'same',
                                       activation = 'relu',

                                       kernel_regularizer =
l2(weight_decay)
                                       )(x)
  x = tf.keras.layers.BatchNormalization()(x)
  x = tf.keras.layers.MaxPool2D(pool_size=(2,2))(x)
  x = tf.keras.layers.Dropout(rate = 0.4)(x)

  x = tf.keras.layers.Flatten()(x)

  x = tf.keras.layers.Dense(1024, activation = 'relu')(x)
  x = tf.keras.layers.Dropout(rate = 0.3)(x)
  x = tf.keras.layers.Dense(512, activation = 'relu')(x)
  x = tf.keras.layers.Dropout(rate = 0.25)(x)
  x = tf.keras.layers.Dense(256, activation = 'relu')(x)
  x = tf.keras.layers.Dropout(rate = 0.2)(x)

  outputs =  tf.keras.layers.Dense(10, activation = 'softmax')(x)

  model = tf.keras.Model(inputs = inputs,outputs=outputs, name =
'cifar-10')


  model.compile(loss = tf.keras.losses.CategoricalCrossentropy(),
                optimizer = tf.keras.optimizers.RMSprop(learning_rate
= lr, decay = lr_decay),
                 metrics=['accuracy'])

  return model

my_model = model_build()
my_model.summary()
```

```
Model: "cifar-10"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 32, 32, 3)]       0

 conv2d (Conv2D)             (None, 32, 32, 32)        896

 batch_normalization (BatchN  (None, 32, 32, 32)       128
 ormalization)

 conv2d_1 (Conv2D)           (None, 32, 32, 32)        9248

 batch_normalization_1 (Batc  (None, 32, 32, 32)       128
 hNormalization)

 max_pooling2d (MaxPooling2D  (None, 16, 16, 32)       0
 )

 dropout (Dropout)           (None, 16, 16, 32)        0

 conv2d_2 (Conv2D)           (None, 16, 16, 64)        18496

 batch_normalization_2 (Batc  (None, 16, 16, 64)       256
 hNormalization)

 conv2d_3 (Conv2D)           (None, 16, 16, 64)        36928

 batch_normalization_3 (Batc  (None, 16, 16, 64)       256
 hNormalization)

 max_pooling2d_1 (MaxPooling  (None, 8, 8, 64)         0
 2D)

 dropout_1 (Dropout)         (None, 8, 8, 64)          0

 conv2d_4 (Conv2D)           (None, 8, 8, 128)         73856

 batch_normalization_4 (Batc  (None, 8, 8, 128)        512
 hNormalization)

 conv2d_5 (Conv2D)           (None, 8, 8, 128)         147584

 batch_normalization_5 (Batc  (None, 8, 8, 128)        512
 hNormalization)

 max_pooling2d_2 (MaxPooling  (None, 4, 4, 128)        0
 2D)

 dropout_2 (Dropout)         (None, 4, 4, 128)         0
```

```
 flatten (Flatten)           (None, 2048)              0

 dense (Dense)               (None, 1024)              2098176

 dropout_3 (Dropout)         (None, 1024)              0

 dense_1 (Dense)             (None, 512)               524800

 dropout_4 (Dropout)         (None, 512)               0

 dense_2 (Dense)             (None, 256)               131328

 dropout_5 (Dropout)         (None, 256)               0

 dense_3 (Dense)             (None, 10)                2570

=================================================================
Total params: 3,045,674
Trainable params: 3,044,778
Non-trainable params: 896
_________________________________________________________________

initial_epochs = 10
hist1 = my_model.fit(train_img,train_label,
                     validation_split = 0.2,
                     epochs = initial_epochs,
                     steps_per_epoch = train_img.shape[0]//batch_size,
                     callbacks = callbacklist)

Epoch 1/10
781/781 [==============================] - 34s 41ms/step - loss:
1.4653 - accuracy: 0.5078 - val_loss: 1.3331 - val_accuracy: 0.5502 -
lr: 0.0010
Epoch 2/10
781/781 [==============================] - 28s 36ms/step - loss:
1.2738 - accuracy: 0.5975 - val_loss: 3.7673 - val_accuracy: 0.3524 -
lr: 0.0010
Epoch 3/10
781/781 [==============================] - 29s 37ms/step - loss:
1.2150 - accuracy: 0.6258 - val_loss: 1.1178 - val_accuracy: 0.6505 -
lr: 0.0010
Epoch 4/10
781/781 [==============================] - 29s 38ms/step - loss:
1.1945 - accuracy: 0.6462 - val_loss: 1.0705 - val_accuracy: 0.6921 -
lr: 0.0010
Epoch 5/10
781/781 [==============================] - 29s 38ms/step - loss:
1.1918 - accuracy: 0.6558 - val_loss: 1.0476 - val_accuracy: 0.6894 -
lr: 0.0010
Epoch 6/10
```

```
781/781 [==============================] - 30s 38ms/step - loss:
1.1638 - accuracy: 0.6660 - val_loss: 1.0965 - val_accuracy: 0.6919 -
lr: 0.0010
Epoch 7/10
781/781 [==============================] - 30s 38ms/step - loss:
1.1662 - accuracy: 0.6739 - val_loss: 1.0272 - val_accuracy: 0.7040 -
lr: 0.0010
Epoch 8/10
781/781 [==============================] - 30s 39ms/step - loss:
1.1712 - accuracy: 0.6761 - val_loss: 1.0993 - val_accuracy: 0.6610 -
lr: 0.0010
Epoch 9/10
781/781 [==============================] - 30s 39ms/step - loss:
1.1701 - accuracy: 0.6848 - val_loss: 0.9889 - val_accuracy: 0.7217 -
lr: 0.0010
Epoch 10/10
670/781 [========================>.....] - ETA: 3s - loss: 1.1918 -
accuracy: 0.6796WARNING:tensorflow:Your input ran out of data;
interrupting training. Make sure that your dataset or generator can
generate at least `steps_per_epoch * epochs` batches (in this case,
7810 batches). You may need to use the repeat() function when building
your dataset.
781/781 [==============================] - 27s 35ms/step - loss:
1.1918 - accuracy: 0.6796 - val_loss: 1.8674 - val_accuracy: 0.5946 -
lr: 0.0010
```

```
total_epochs = initial_epochs + 10
hist2 = my_model.fit(train_img,train_label,
                     validation_split = 0.2,
                     initial_epoch = hist1.epoch[-1],
                     epochs = total_epochs,
                     steps_per_epoch = train_img.shape[0]//batch_size,
                     callbacks = callbacklist)
```

```
Epoch 10/20
781/781 [==============================] - 33s 42ms/step - loss:
1.1830 - accuracy: 0.6818 - val_loss: 0.9694 - val_accuracy: 0.7319 -
lr: 0.0010
Epoch 11/20
781/781 [==============================] - 31s 40ms/step - loss:
1.1751 - accuracy: 0.6903 - val_loss: 0.9360 - val_accuracy: 0.7476 -
lr: 0.0010
Epoch 12/20
781/781 [==============================] - 30s 38ms/step - loss:
1.1881 - accuracy: 0.6914 - val_loss: 1.2559 - val_accuracy: 0.7215 -
lr: 0.0010
Epoch 13/20
781/781 [==============================] - 30s 38ms/step - loss:
1.1895 - accuracy: 0.6917 - val_loss: 0.9428 - val_accuracy: 0.7569 -
lr: 0.0010
Epoch 14/20
```

```
781/781 [==============================] - 32s 41ms/step - loss:
1.1868 - accuracy: 0.6939 - val_loss: 1.1534 - val_accuracy: 0.6888 -
lr: 0.0010
Epoch 15/20
781/781 [==============================] - 32s 41ms/step - loss:
1.2129 - accuracy: 0.6920 - val_loss: 0.9613 - val_accuracy: 0.7554 -
lr: 0.0010
Epoch 16/20
781/781 [==============================] - 32s 41ms/step - loss:
1.1898 - accuracy: 0.6978 - val_loss: 1.3250 - val_accuracy: 0.6907 -
lr: 0.0010
Epoch 17/20
781/781 [==============================] - 32s 41ms/step - loss:
0.9983 - accuracy: 0.7567 - val_loss: 0.8306 - val_accuracy: 0.7977 -
lr: 2.0000e-04
Epoch 18/20
781/781 [==============================] - 32s 41ms/step - loss:
0.9602 - accuracy: 0.7699 - val_loss: 0.8427 - val_accuracy: 0.7940 -
lr: 2.0000e-04
Epoch 19/20
781/781 [==============================] - 32s 40ms/step - loss:
0.9375 - accuracy: 0.7754 - val_loss: 0.8336 - val_accuracy: 0.7909 -
lr: 2.0000e-04
Epoch 20/20
659/781 [========================>.....] - ETA: 4s - loss: 0.9241 -
accuracy: 0.7780WARNING:tensorflow:Your input ran out of data;
interrupting training. Make sure that your dataset or generator can
generate at least `steps_per_epoch * epochs` batches (in this case,
8591 batches). You may need to use the repeat() function when building
your dataset.
781/781 [==============================] - 27s 34ms/step - loss:
0.9241 - accuracy: 0.7780 - val_loss: 0.8188 - val_accuracy: 0.8016 -
lr: 2.0000e-04
```

```
total_epochs = initial_epochs + 20
hist3 = my_model.fit(train_img,train_label,
                     validation_split = 0.2,
                     initial_epoch = hist2.epoch[-1],
                     epochs = total_epochs,
                     steps_per_epoch = train_img.shape[0]//batch_size,
                     callbacks = callbacklist)
```

```
Epoch 20/30
781/781 [==============================] - 33s 42ms/step - loss:
0.9181 - accuracy: 0.7803 - val_loss: 0.7973 - val_accuracy: 0.8123 -
lr: 2.0000e-04
Epoch 21/30
781/781 [==============================] - 32s 41ms/step - loss:
0.9212 - accuracy: 0.7800 - val_loss: 0.8542 - val_accuracy: 0.7973 -
lr: 2.0000e-04
Epoch 22/30
```

```
781/781 [==============================] - 32s 40ms/step - loss:
0.9126 - accuracy: 0.7799 - val_loss: 0.7931 - val_accuracy: 0.8104 -
lr: 2.0000e-04
Epoch 23/30
781/781 [==============================] - 31s 40ms/step - loss:
0.9052 - accuracy: 0.7817 - val_loss: 0.8075 - val_accuracy: 0.8022 -
lr: 2.0000e-04
Epoch 24/30
781/781 [==============================] - 31s 40ms/step - loss:
0.9105 - accuracy: 0.7832 - val_loss: 0.7963 - val_accuracy: 0.8064 -
lr: 2.0000e-04
Epoch 25/30
781/781 [==============================] - 31s 39ms/step - loss:
0.9031 - accuracy: 0.7853 - val_loss: 0.7891 - val_accuracy: 0.8152 -
lr: 2.0000e-04
Epoch 26/30
781/781 [==============================] - 31s 39ms/step - loss:
0.8854 - accuracy: 0.7888 - val_loss: 0.8012 - val_accuracy: 0.8064 -
lr: 2.0000e-04
Epoch 27/30
781/781 [==============================] - 30s 38ms/step - loss:
0.8906 - accuracy: 0.7863 - val_loss: 0.7983 - val_accuracy: 0.8172 -
lr: 2.0000e-04
Epoch 28/30
781/781 [==============================] - 29s 38ms/step - loss:
0.9016 - accuracy: 0.7866 - val_loss: 0.8242 - val_accuracy: 0.8014 -
lr: 2.0000e-04
Epoch 29/30
781/781 [==============================] - 30s 39ms/step - loss:
0.8866 - accuracy: 0.7901 - val_loss: 0.8133 - val_accuracy: 0.7960 -
lr: 2.0000e-04
Epoch 30/30
659/781 [========================>.....] - ETA: 4s - loss: 0.8818 -
accuracy: 0.7899WARNING:tensorflow:Your input ran out of data;
interrupting training. Make sure that your dataset or generator can
generate at least `steps_per_epoch * epochs` batches (in this case,
8591 batches). You may need to use the repeat() function when building
your dataset.
781/781 [==============================] - 27s 35ms/step - loss:
0.8818 - accuracy: 0.7899 - val_loss: 0.8284 - val_accuracy: 0.8123 -
lr: 2.0000e-04
```

```
my_model.save("/content/drive/MyDrive/Colab Notebooks/week
6/newmodel.h5")
```

```
total_epochs = initial_epochs + 30
hist4 = my_model.fit(train_img,train_label,
                     validation_split = 0.2,
                     initial_epoch = hist3.epoch[-1],
                     epochs = total_epochs,
```

```
                    steps_per_epoch = train_img.shape[0]//batch_size,
                    callbacks = callbacklist)
```

```
Epoch 30/40
781/781 [==============================] - 30s 39ms/step - loss:
0.8352 - accuracy: 0.8047 - val_loss: 0.7798 - val_accuracy: 0.8142 -
lr: 1.0000e-04
Epoch 31/40
781/781 [==============================] - 29s 37ms/step - loss:
0.8344 - accuracy: 0.8036 - val_loss: 0.8000 - val_accuracy: 0.8209 -
lr: 1.0000e-04
Epoch 32/40
781/781 [==============================] - 28s 36ms/step - loss:
0.8371 - accuracy: 0.8035 - val_loss: 0.8024 - val_accuracy: 0.8161 -
lr: 1.0000e-04
Epoch 33/40
781/781 [==============================] - 28s 36ms/step - loss:
0.8447 - accuracy: 0.8048 - val_loss: 0.7972 - val_accuracy: 0.8141 -
lr: 1.0000e-04
Epoch 34/40
781/781 [==============================] - 30s 39ms/step - loss:
0.8337 - accuracy: 0.8046 - val_loss: 0.8072 - val_accuracy: 0.8147 -
lr: 1.0000e-04
Epoch 35/40
781/781 [==============================] - 32s 41ms/step - loss:
0.8208 - accuracy: 0.8099 - val_loss: 0.7864 - val_accuracy: 0.8157 -
lr: 1.0000e-04
Epoch 36/40
781/781 [==============================] - 31s 39ms/step - loss:
0.8437 - accuracy: 0.8064 - val_loss: 0.8098 - val_accuracy: 0.8158 -
lr: 1.0000e-04
```

```
my_model.save("/content/drive/MyDrive/Colab Notebooks/week
6/finalmodel.h5")
```

```
# !kill 525
```

Tensorboard

```
%reload_ext tensorboard
```

```
%tensorboard --logdir '/content/drive/MyDrive/Colab Notebooks/week
6/logs2'
```

```
<IPython.core.display.Javascript object>
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Hyper parameters tuning

tried using keras tuner which took more time and accuracy was around 60%

and did tune hyperparameters manually which resulted in 75% validation accuracy

both ways the model did not overfit.

Observation which gave us good results:
weight_decay = 1e-4
batch_size = 64
lr = 0.001
lr_decay = 1e-6
num of filters = 32 for first 2 conv layers, 64 for next 2 conv layers, 128 for next 2 conv layers
kernel size = 3x3
regularizer = l2
dropout = 0.2 after first 2 conv layers,0.3 after next 2 conv layers,0.4 after last 2 conv layers
the model was in the structure

Model: "cifar-10"

_____

Layer (type) Output Shape Param #

==========

input_1 (InputLayer) [(None, 32, 32, 3)] 0

conv2d (Conv2D) (None, 32, 32, 32) 896
batch_normalization (Batch Normalization) (None, 32, 32, 32) 128

conv2d_1 (Conv2D) (None, 32, 32, 32) 9248
batch_normalization_1 (Batch Normalization) (None, 32, 32, 32) 128

max_pooling2d (MaxPooling2D) (None, 16, 16, 32) 0

dropout (Dropout) (None, 16, 16, 32) 0
conv2d_2 (Conv2D) (None, 16, 16, 64) 18496
batch_normalization_2 (Batch Normalization) (None, 16, 16, 64) 256

conv2d_3 (Conv2D) (None, 16, 16, 64) 36928
batch_normalization_3 (Batch Normalization) (None, 16, 16, 64) 256

max_pooling2d_1 (MaxPooling2D) (None, 8, 8, 64) 0

dropout_1 (Dropout) (None, 8, 8, 64) 0
conv2d_4 (Conv2D) (None, 8, 8, 128) 73856
batch_normalization_4 (BatchNormalization) (None, 8, 8, 128) 512

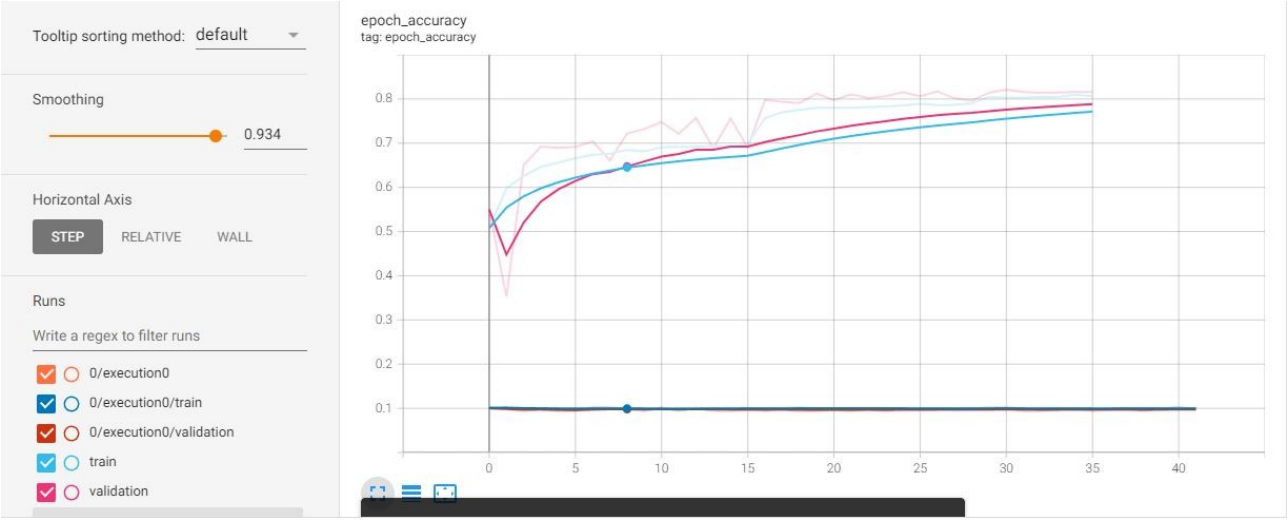
conv2d_5 (Conv2D) (None, 8, 8, 128) 147584
batch_normalization_5 (BatchNormalization) (None, 8, 8, 128) 512

max_pooling2d_2 (MaxPooling2D) (None, 4, 4, 128) 0

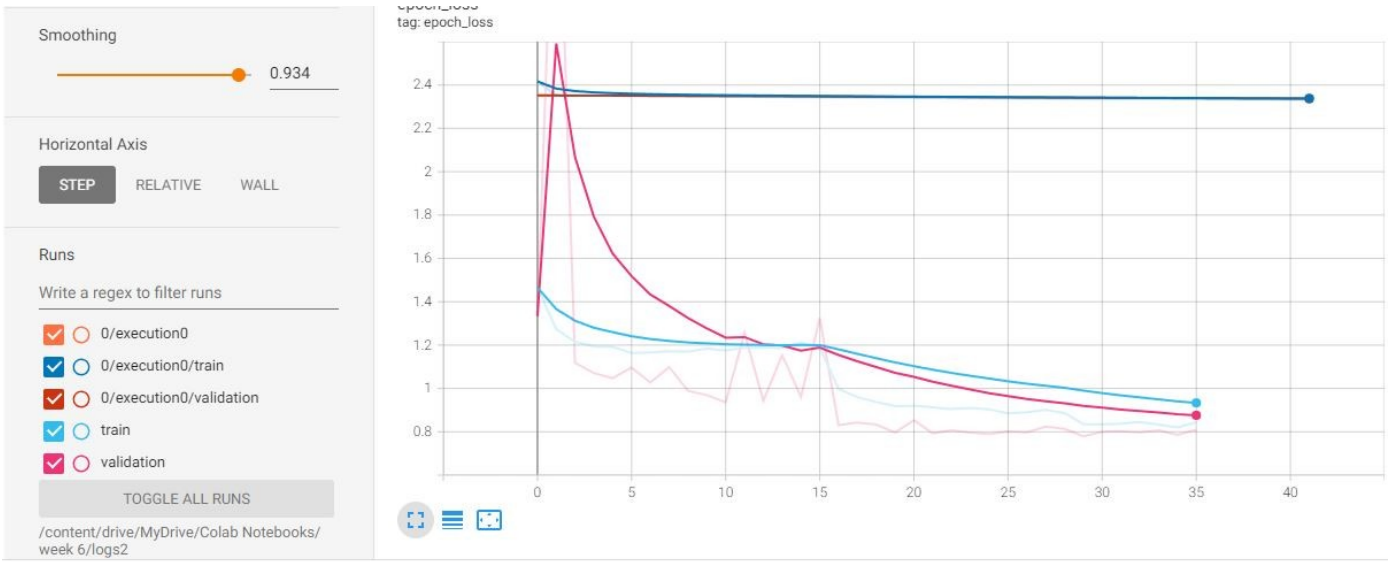
dropout_2 (Dropout) (None, 4, 4, 128) 0

flatten (Flatten) (None, 2048) 0
dense (Dense) (None, 1024) 2098176
dropout_3 (Dropout) (None, 1024) 0
dense_1 (Dense) (None, 512) 524800
dropout_4 (Dropout) (None, 512) 0
dense_2 (Dense) (None, 256) 131328
dropout_5 (Dropout) (None, 256) 0
dense_3 (Dense) (None, 10) 2570
===
====
Total params: 3,045,674
Trainable params: 3,044,778
Non-trainable params: 896
_____

Accuracy Graph



Loss Graph


Smoothing
0.934
Horizontal Axis
STEP
RELATIVE
WALL
Runs
Write a regex to filter runs
0/execution0
0/execution0/train
0/execution0/validation
train
validation
TOGGLE ALL RUNS
/content/drive/MyDrive/Colab Notebooks/week 6/logs2
tag: epoch_loss
2.4
2.2
2
1.8
1.6
1.4
1.2
1
0.8
0
5
10
15
20
25
30
35
40